Tipos Abstratos de Dados (TADs)

Exercícios - escolha da implementação

Exercício

Analise o trecho de código abaixo:

```
def extrai_metade_menores(lista_dados):
    n = len(lista_dados)
    colecao = cria_inicializada(lista_dados)
    menores = []
    for _ in range(n // 2):
        menores.append(colecao.remove_menor())
    return menores
```

Tarefa:

1. Conte a inicialização e quantas vezes a remoção do menor elemento é executada.
2. Determine a estrutura que minimiza o tempo total de execução.
3. Defina a complexidade total do algoritmo.

Gabarito: TAD Fila de Prioridade

- **Contagem de operações:** 1 chamada para `cria_inicializada` recebendo n elementos e $n/2$ chamadas da operação `remove_menor`.
- **Estrutura escolhida:** TAD Fila de Prioridade implementado com **Heap Binária** (`heapq`).
- **Por que escolher esta estrutura?**
  - O algoritmo precisa apenas construir a base e extrair os menores elementos.
  - A inicialização com Heap (`heapify`) constrói a estrutura em tempo linear $O(n)$, enquanto uma árvore balanceada levaria $O(n \log n)$.
  - A remoção do menor elemento é rápida e custa $O(\log n)$.
- **Complexidade Total:** Inicialização em $O(n) + \frac{n}{2} \times O(\log n) = \mathbf{O(n \log n)}$.
  - A implementação com árvore balanceada custaria $O(n \log n)$ para inicializar e $O(\log n)$ para cada remoção, resultando em $O(n \log n)$ também.
  - Porém, a heap binária tem constantes menores, e tem implementação mais simples.

Exercício

Analise o seguinte fluxo de movimentações de estoque:

```
def processa_estoque(entradas, saidas):
    colecao = cria_colecao_vazia()
    for item in entradas: # itens podem se repetir
        colecao.insere_elemento(item)
    for item in saidas:
        colecao.remove_elemento(item)
    return colecao.tamanho_total()
```

Tarefa:

1. Conte o número de chamadas de operações para n entradas e m saídas.
2. Qual TAD, e respectiva implementação, são mais naturais para esta tarefa?
3. Forneça a complexidade total do algoritmo.
4. Qual seria a complexidade se `colecao` for uma lista encadeada (deque)?

Gabarito: TAD Multiconjunto

- **Contagem de operações:**
  - `insere_elemento`: n vezes
  - `remove_elemento`: m vezes
  - `tamanho_total`: 1 vez
- **Estrutura escolhida:** TAD Multiconjunto (*Bag*) implementado com **Tabela Hash com contadores** (`multiset`).
- **Por que escolher esta estrutura?**
  - O multiconjunto permite armazenar elementos repetidos guardando cada item como uma chave e sua quantidade como o valor.
  - Inserir ou remover apenas incrementa ou decrementa a contagem do item em $O(1)$.
  - A consulta do tamanho total também custa $O(1)$.
- **Complexidade Total:** $n \times O(1) + m \times O(1) + O(1) = \mathbf{O(n + m)}$.
- **Complexidade com Lista Encadeada:** a inserção seria $O(1)$, mas a remoção exigiria percorrer a lista para encontrar o elemento, resultando em $O(m \cdot n)$ no pior caso. A consulta do tamanho total seria $O(1)$. Portanto, $O(n + m \cdot n) = O(m \cdot n)$.

Exercício

Analise a função abaixo:

```
def processa_dados(n):
    colecao = cria_colecao_vazia()
    for i in range(n):
        colecao.insere_no_final(i)

    soma = 0
    for i in range(n):
        indice = (i * 3) % n
        soma += colecao.acessa_posicao(indice)

    return soma
```

Tarefa:

1. Conte quantas vezes cada operação da coleção é chamada em função de n.
2. Escolha a melhor estrutura vista em aula para obter o menor tempo de execução.
3. Calcule a complexidade total do algoritmo.

Gabarito: TAD Lista

- **Contagem de operações:**
  - `cria_colecao_vazia`: 1 vez
  - `insere_no_final`: n vezes
  - `acessa_posicao`: n vezes
- **Estrutura escolhida:** TAD Lista implementado com **Array Dinâmico** (`list` do Python).
- **Por que escolher esta estrutura?**
  - A inserção no final custa $O(1)$ amortizado.
  - O acesso direto por índice i leva tempo $O(1)$.
  - Se usássemos uma lista encadeada (ou Deque), acessar uma posição arbitrária i custaria $O(n)$, o que deixaria o algoritmo em $O(n^2)$.
- **Complexidade Total:** $1 \times O(1) + n \times O(1) + n \times O(1) = \mathbf{O(n)}$.

Exercício

Analise o algoritmo de contagem de votos:

```
1 def conta_votos(votos):
2     colecao = cria_colecao_vazia()
3     for candidato in votos:
4         if colecao.pertence(candidato):
5             qtd = colecao.acessa_por_chave(candidato)
6             colecao.altera_chave(candidato, qtd + 1)
7         else:
8             colecao.insere_chave(candidato, 1)
9     return colecao
```

Tarefa:

1. Conte quantas operações são realizadas para uma entrada com n votos.
2. Indique qual estrutura de dados abordada em aula deve ser usada.
3. Calcule a complexidade média total do algoritmo.

Gabarito: TAD Dicionário

- **Contagem de operações:** O laço roda n vezes. Em cada volta, chama `pertence` (1 vez) e faz uma inserção (`insere_chave`) ou uma leitura com atualização (`acessa_por_chave` e `altera_chave`).
- **Estrutura escolhida:** TAD Dicionário (Mapa) implementado com **Tabela Hash** (`dict`).
- **Por que escolher esta estrutura?**
  - Precisamos associar chaves únicas a valores (contadores).
  - Na Tabela Hash, as operações de buscar (`pertence`), acessar e atualizar chaves executam em tempo médio constante, ou seja, $O(1)$.
- **Complexidade Total:** Com n iterações custando tempo médio $O(1)$, o total é $\mathbf{O(n)}$.

Exercício

Analise o algoritmo que calcula a soma dos pesos de todas as arestas do grafo:

```
1  def soma_pesos(vertices, arestas):
2      G = cria_grafo_vazio()
3      for u in vertices:
4          G.insere_vertice(u)
5      for u, v, w in arestas:
6          G.insere_aresta(u, v, w)
7
8      total = 0
9      for u in G.iterar_vertices():
10         for v in G.iterar_vizinhos(u):
11             total += G.peso(u, v)
12     return total
```

Tarefa:

1. Conte o total de execuções de cada operação ($n = |\text{vértices}|$, $m = |\text{arestas}|$).
2. Calcule a complexidade total do algoritmo.

Gabarito: TAD Grafo

- **Contagem de operações:** Inserem-se n vértices e m arestas. O primeiro laço itera pelos n vértices; o laço interno percorre apenas os vizinhos de cada vértice u, rodando d_u vezes (grau do vértice u).
- **Estrutura escolhida:** TAD Grafo implementado como Lista de Adjacências via **Dicionário de Adjacências** (dicionário de dicionários).
- **Por que escolher esta estrutura?**
  - Se usássemos Matriz de Adjacência, percorrer os vizinhos de um vértice exigiria olhar a linha inteira da matriz, gastando $O(n)$ por vértice e $O(n^2)$ no total.
  - No dicionário de adjacências, olhar os vizinhos de u custa exatamente $O(d_u)$.
  - A soma dos graus de todos os vértices é igual ao dobro do número de arestas: $\sum_u O(d_u) = O(m)$.
- **Complexidade Total:** Percorrer todos os vértices e vizinhos resulta em $\mathbf{O(n + m)}$.